

Top-of-table AI Search: Dual-Mode Design, Ready To Build

Below is a complete, developer-ready plan for a single command bar that sits above the master data table and delivers the two functions you want. It is lean, fast, auditable, and built for million-item scale.

1. Where it lives and how it behaves

- A single command bar sits above the master table
- Two modes inside the bar: Filter and Evidence
- Cmd K opens the bar, Enter runs, Esc cancels
- A compact Query Plan pill shows what the system understood before execution, clicking it reveals all parsed facets as editable chips

2. Function 1: Intelligent filtering from natural language

Goal

Turn plain English into precise table filters without touching the AI model that reads documents.

Example input

“Show me all emails from the client to United Living between January 2021 and February 2022”

What happens

1. Detect intent as Filter
2. Extract facets
 - Parties and aliases resolved via the case party dictionary created at configuration
 - Direction A to B
 - Channel equals email
 - Date range normalised to UTC, displayed in user locale
3. Render chips before running
 - From equals Client Org

- To equals United Living
- Type equals Email
- Date equals 01 Jan 2021 to 28 Feb 2022

4. On Apply, run a server-side query that updates the table and persists the saved search

Parser contract

Input: free text, party dictionary, document types, timezone

Output: structured facets with confidence per facet, plus any unresolved tokens for user confirmation

Edge handling

- If “client” is ambiguous, show a two-option chooser in the bar
- If the period is malformed, prompt with a calendar chip rather than failing
- If direction is missing, default to any direction and display the default in the Query Plan pill

Pseudocode sketch

```
python Copy

def parse_to_facets(text, party_dict, tz):
    facets = {}
    entities = ner_extract(text) # parties, dates, channels, directions, keywords
    facets["type"] = "email" if "email" in entities.channels else None
    a, b = resolve_parties(entities.parties, party_dict) # returns from, to, or None
    facets["from"], facets["to"] = a, b
    facets["direction"] = "A_TO_B" if a and b else None
    facets["date_range"] = normalize_dates(entities.dates, tz)
    facets["contains"] = entities.keywords # optional
    return facets, unresolved_tokens(entities)
```

4. Table and drawer behaviour

- Filter mode updates the table only and writes a human readable line under the bar, for example From Client to UL, Email, Jan 2021 to Feb 2022
- Evidence mode leaves the table intact and opens a right drawer with the ranked results and explainers
- Both modes keep the Query Plan pill visible and editable so users can correct any facet instantly

5. Performance, safety, and audit

- A Research plan will not execute unless at least period or parties is set, the bar asks for one more answer
- Index level permissions ensure no leaks

- Every plan and its result count is logged with user, time, and case, including the exact facets used
- Large sets are paginated, sorted by sent time and authored score for emails

6. Copy and micro interaction, exactly as users will see it

Filter mode placeholder

Type a filter, for example emails from the client to United Living in Q1 2022

Evidence mode placeholder

Ask a factual question, for example find everything about the loading bay

Chip prompts after “loading bay”

- Select a period, Jan to Mar 2022, Apr to Jun 2022, Custom
- Select people, UL, LJJ, NHBC, Site Logistics, Add others
- Select scope, emails, attachments, all

If construction add in is on

- Select programme, Activity 210 Foundation, Window W3

Explainer line for a hit

Matched on phrase “loading bay”, authored by Site Logistics, 12 Feb 2022, within selected period

7. Acceptance criteria your developer can test

- Free text “emails from client to United Living between Jan 2021 and Feb 2022” produces four chips that match those facets and yields the same result set as manual filters
- Evidence query “loading bay” will not execute until at least one period or one stakeholder is chosen
- With construction add in active, adding Activity 210 reduces the result set and promotes messages that mention that activity, verified by inspection
- Every executed plan appears in the audit log with facets, user, timestamp, result count, and duration
- A saved search reproduces the same result set when reloaded

8. Minimal build plan

Week one

- React command bar with two modes, chips, Query Plan pill, keyboard bindings
- Natural language to facets parser for dates, parties, direction, type
- Server-side filter execution and chip persistence

Week two

- Evidence mode slot filling with chip questions powered by corpus histograms and party dictionaries
- Hybrid retrieval service with lexical plus vector search and a simple reranker
- Right drawer results with explainers and adjustable chips

Week three

- Telemetry and audit events
- Saved searches and shareable links
- Edge cases and language variants, for example “from UL to client”, “last winter”, “past three months”

9. Why this gives you an edge

- Users type what they mean, then see exactly what the system understood, which prevents silent errors
- The assistant never trawls blindly across the corpus, it insists on one or two context answers, which keeps it fast and accurate
- It is entirely auditable and repeatable, which is critical in adjudication and disclosure
- It scales, because search engines do the heavy lifting and the model only scores a small candidate set